

Config, Logging & Observability Cheatsheet

Lessons: [23 — Config, Logging & Observability](#) · [39 — Distributed Tracing](#)

Examples: [39](#)

Covers: env config, `log/slog`, request logging, health checks, metrics, OpenTelemetry spans

Legend: `[*]` = real API that the lessons have not covered yet

CONFIG: the 12-factor shape

```
type Config struct { Port int; DatabaseURL string; LogLevel string }
func Load() (*Config, error)      one function, called once in main
os.Getenv("PORT")                  "" when unset
os.LookupEnv("PORT")               -> (value, ok): distinguishes unset from empty
strconv.Atoi(os.Getenv(...))      parse, and FAIL if it's malformed
fail fast at startup               a missing DATABASE_URL should stop the process
defaults for local dev             sensible values, never for secrets
never commit secrets              env vars or a secret manager, not the repo
flag for one-off overrides         env wins in containers; flags win on a laptop
caarlos0/env                      [*] struct-tag based env parsing
kelseyhightower/envconfig         [*] the older equivalent
(config is read ONCE and passed down – never call os.Getenv deep in a handler)
```

slog: the API

```
slog.Info("user created", "user_id", id, "tenant", t)  alternating key/value
slog.Debug / slog.Info / slog.Warn / slog.Error        the four levels
slog.String("k", v) / Int / Int64 / Bool / Float64 / Duration / Time / Any
                                                         typed attrs: faster, and a typo won't compile
slog.New(slog.NewJSONHandler(os.Stdout, opts))         production: JSON to stdout
slog.New(slog.NewTextHandler(os.Stderr, opts))         local: key=value
slog.SetDefault(logger)                               the package-level slog.Info now uses it
logger.With("request_id", id)                         a child logger that carries fixed attrs
logger.InfoContext(ctx, "msg", ...)                   pass ctx so handlers can enrich
slog.Group("user", "id", 1, "role", "admin")          [*] nested attributes
&slog.HandlerOptions{
  Level: slog.LevelDebug,          the minimum level to emit
  AddSource: true,                 [*] file:line of the call site
  ReplaceAttr: func(...)           [*] rewrite/redact attributes centrally
}
slog.LevelVar                      [*] change the level at runtime, atomically
(log to STDOUT and let the platform collect it – never to a file you rotate)
```

LOGGING RULES

one event, one line	structured, machine-parsable
message is a constant	"user created", not "user 42 created"
the variables are ATTRS	so you can filter and aggregate on them
levels mean something	Error = a human must look; Warn = degraded
log at the boundary	not in every layer of the call stack
log OR return the error	never both – that's how one failure becomes five lines
never log secrets	passwords, tokens, full request bodies, card numbers
redact centrally	ReplaceAttr, so nobody has to remember
request_id on every line	the thread that ties a request together
(if a log line can't be filtered on in production, it is decoration)	

REQUEST LOGGING MIDDLEWARE

generate a request id	<code>uuid.NewString()</code> or the incoming <code>X-Request-ID</code>
put it in the context	<code>ctx = context.WithValue(ctx, reqIDKey{}, id)</code>
echo it back	<code>w.Header().Set("X-Request-ID", id)</code>
wrap the <code>ResponseWriter</code>	to capture the status code and bytes written
log once, after <code>next()</code>	method, path, status, duration, bytes, <code>request_id</code>
skip <code>/healthz</code>	or your logs are 90% health checks
propagate it downstream	send <code>X-Request-ID</code> on every outbound call

HEALTH CHECKS

<code>GET /healthz</code>	LIVENESS: is the process alive? return 200, no I/O
<code>GET /readyz</code>	READINESS: can it serve? check the DB, return 503 if not
<code>db.PingContext(ctx)</code>	with a short timeout – 1s, not the default
flip <code>/readyz</code> to 503 first	then start draining, so the LB stops sending traffic
<code>/metrics</code>	Prometheus scrape endpoint
never auth <code>/healthz</code>	the platform can't log in
liveness must not check deps	a slow DB should not restart your process

METRICS (Prometheus shape) [*]

Counter	only goes up: <code>requests_total</code> , <code>errors_total</code>
Gauge	goes up and down: <code>queue_depth</code> , <code>connections_in_use</code>
Histogram	distribution: <code>request_duration_seconds</code>
Summary	client-side quantiles (prefer Histogram)
labels	method, route, status – LOW cardinality only
never label with <code>user_id</code>	one time series per user will kill the store
<code>prometheus/client_golang</code>	the library; <code>promhttp.Handler()</code> for <code>/metrics</code>
the four golden signals	latency, traffic, errors, saturation
RED for services	Rate, Errors, Duration
USE for resources	Utilization, Saturation, Errors

TRACING: the model

trace	one request's whole journey, across services
span	one unit of work inside it, with a start and end
trace id	the same for every span in the trace
span id / parent span id	the tree structure
attributes	key/value on a span (<code>http.method</code> , <code>db.statement</code>)
events	a timestamped log line attached to a span
context propagation	the trace id travels in HTTP headers (<code>traceparent</code>)
W3C Trace Context	the standard header format
sampling	record a percentage; head-based is the common choice
(logs tell you WHAT happened, metrics tell you HOW OFTEN, traces tell you WHERE)	

OpenTelemetry API [*]

<code>otel.Tracer("service-name")</code>	get a tracer
<code>ctx, span := tracer.Start(ctx, "GetUser")</code>	start a span; ctx now carries it
<code>defer span.End()</code>	always, or the span never closes
<code>span.SetAttributes(attribute.String("user.id", id))</code>	
<code>span.RecordError(err)</code>	attach the error
<code>span.SetStatus(codes.Error, "failed")</code>	mark the span as failed
<code>trace.SpanFromContext(ctx)</code>	the current span, anywhere down the stack
<code>otelhttp.NewHandler(h, "server")</code>	auto-instrument an HTTP server
<code>otelhttp.NewTransport(rt)</code>	auto-instrument an HTTP client
<code>otel.SetTracerProvider(tp)</code>	wire it up once in main
<code>tp.Shutdown(ctx)</code>	flush buffered spans before exit
<code>propagation.TraceContext{}</code>	inject/extract the traceparent header
(pass ctx everywhere or the trace breaks at the first function that drops it)	

CORRELATING THE THREE PILLARS

put <code>trace_id</code> in every log line	jump from a log to the trace
exemplars on histograms	[*] jump from a metric spike to a trace
same <code>request_id</code> across services	one id, propagated
structured logs + trace id	the cheapest observability that actually works
(start with logs + request id; add metrics; add tracing when you have >2 services)	

TRAPS & MEMORIZE

<code>os.Getenv</code> deep in the code	untestable and undiscoverable; load config once
config read per request	silently slow, and impossible to validate
logging to a file	containers are ephemeral; log to stdout
<code>fmt.Println</code> for logs	no level, no fields, no timestamp
<code>log.Fatal</code> in a library	it calls <code>os.Exit</code> – never do this outside main
logging and returning err	the same failure appears at every layer
high-cardinality labels	<code>user_id/request_id</code> in metrics blows up the TSDB
liveness that checks the DB	a DB blip restarts every replica at once
readiness that checks nothing	traffic arrives before the pool is warm
<code>span.End()</code> without <code>defer</code>	an early return leaks the span
dropping ctx	the trace ends there, and so does cancellation
sampling at 100%	fine in dev, expensive and noisy in production